Confidential Transfer

> ***Nox Protocol — Developer Guide*** *Arbitrum Sepolia Testnet (chainId: 421614)*

Overview

The **Confidential Transfer** sends confidential tokens (e.g. cRLC) from one address to another while keeping the **amount encrypted**. Only the sender and recipient can see the transfer amount — it is never exposed on-chain.

The recipient address itself is public (visible on-chain), but the transferred amount is encrypted using the Nox HandleClient before being submitted to the contract.

Use case: A user sends 25 cRLC to a business partner. The transaction appears on Arbiscan, but the amount field shows an encrypted handle — not the actual value.

Architecture

Component	Path / Address
Hook	`hooks/use-confidential-transfer.ts`
Modal	`components/modals/transfer-modal.tsx`
HandleClient	`@iexec-nox/handle` (client-side only)
cToken ABI	`lib/confidential-token-abi.ts`
Gas Utility	`lib/gas.ts`
cRLC Contract	`0x271f46e78f2fe59817854dabde47729ac4935765`

Step-by-Step Flow

```
User                HandleClient           cToken (cRLC)
  |                     |                      |
  |-- 1. encryptInput -->|                      |
  |<-- {handle, proof} --|                      |
  |                     |                      |
  |-- 2. confidentialTransfer(to, handle, proof) -->|
  |<--- tx receipt ---------------------------------|
  |                     |                      |
  | Sender:   cRLC -= N (encrypted)              |
  | Receiver: cRLC += N (encrypted)              |
```

Step 0 — Validation

Before initiating the transfer, the hook validates:

```
// Wallet must be connected
if (!address) throw new Error("Wallet not connected");

// Recipient must be a valid Ethereum address
import { isAddress } from "viem";
if (!isAddress(recipient)) throw new Error("Invalid recipient address");

// Amount must be positive
if (parsedAmount <= 0n) throw new Error("Amount must be greater than 0");
```

> ***Always use `isAddress()` from viem*** *— never use regex for address validation.*

Step 1 — Encrypt Input

The transfer amount is encrypted client-side using the Nox HandleClient. This ensures the amount is never visible in the transaction calldata.

```
import { HandleClient } from "@iexec-nox/handle";

const handleClient = new HandleClient({
  chainId: 421614,
  account: walletClient.account,
});

const { handle, handleProof } = await handleClient.encryptInput(
  parsedAmount,    // BigInt — amount in wei
  "uint256",       // Solidity type
  cTokenAddress    // Contract that will consume this input
);
```

Returns:

Field	Type	Description
`handle`	`bytes32`	Encrypted reference to the amount
`handleProof`	`bytes`	Zero-knowledge proof of correct encryption

> ***Client-side only:*** *HandleClient uses Web Crypto API — never import in SSR.*

Step 2 — Confidential Transfer

Calls `confidentialTransfer()` on the confidential token contract.

```
// Called on the cToken contract (e.g. cRLC)
function confidentialTransfer(
    address to,
    bytes32 encryptedAmount,
    bytes calldata inputProof
) external;
```

Parameters:

Param	Value	Description
`to`	Recipient address	Cleartext — visible on-chain
`encryptedAmount`	`handle` from Step 1	Encrypted amount reference
`inputProof`	`handleProof` from Step 1	Proof of correct encryption

Implementation:

```
const overrides = await estimateGasOverrides(publicClient);

const txHash = await writeContractAsync({
  address: confidentialAddress,
  abi: confidentialTokenAbi,
  functionName: "confidentialTransfer",
  args: [recipient, handle, handleProof],
  ...overrides,
});

await waitForTransactionReceipt(config, { hash: txHash });
```

State Machine

```
idle → encrypting → transferring → confirmed
                          ↘
                         error
```

State	UI	Description
`idle`	Form visible	User selects token, enters amount + recipient
`encrypting`	Spinner	HandleClient encrypting amount
`transferring`	Progress bar, spinner	Transfer tx pending
`confirmed`	Success badge + Arbiscan link	Transaction confirmed
`error`	Error message + retry button	Any step failed

Transaction Info Summary (Modal UI)

The modal displays a summary before confirmation:

Field	Value	Notes
Recipient	`0xAbC...dEf` → Encrypted Hash	Shows address is public but amount is private
Token	cRLC	Selected confidential token

Network Fee	Dynamic (via `useEstimatedFee`)	~200k gas units estimated

Gas Estimation

Estimated gas limit: ~200,000 gas units

```
// Dynamic fee estimation in the modal
const { fee, isLoading } = useEstimatedFee(200_000);
// Returns fee in ETH (e.g. "0.000042 ETH")
```

All transactions use the 20% EIP-1559 buffer (see `lib/gas.ts`).

What Happens On-Chain

After a successful confidential transfer:

1. **Sender's balance handle changes** — the old handle is invalidated, a new one is created
2. **Recipient's balance handle changes** — if they already had cRLC, their handle is updated
3. **Event emitted** — `ConfidentialTransfer` event (amount is encrypted in the event logs too)
4. **Arbiscan** — Transaction visible, but amount field shows the encrypted handle, not the cleartext value

---

Important Notes

1. **Recipient address is public** — Only the amount is encrypted. The recipient's address is visible on-chain in the transaction calldata and event logs.

2. **No approval needed** — Unlike wrap, transfer doesn't require a prior `approve()` call. The user is spending their own cTokens.

3. **Handle changes for both parties** — After the transfer, both sender and recipient must re-read `confidentialBalanceOf` to get their updated handles.

4. **Viewer access invalidated** — If the sender had granted viewer access (via `addViewer`) on their old handle, the viewer must be re-granted access on the new handle after the transfer.

5. **Single transaction** — Unlike unwrap (2-step), transfer is a single on-chain transaction.

6. **Client-side encryption** — The `encryptInput()` call happens in the browser. The encrypted handle + proof are sent to the contract which verifies them in the TEE.

Error Handling

Error	Context	Recovery
Invalid recipient	Validation	Fix address in input field
User rejected tx	Step 2	Reset and retry
Encrypt failed	Step 1	Check HandleClient initialization

Insufficient balance	Step 2	Wrap more tokens first
Recipient is zero address	Validation	Enter valid recipient

Code Section (Dev Mode)

```
// Step 1: Encrypt the transfer amount
handleClient.encryptInput(amount, "uint256", cRLC_address);

// Step 2: Execute confidential transfer
cRLC.confidentialTransfer(recipientAddress, encryptedHandle, inputProof);
```